Manual Técnico — SmartStock AI

Oct 22, 2026

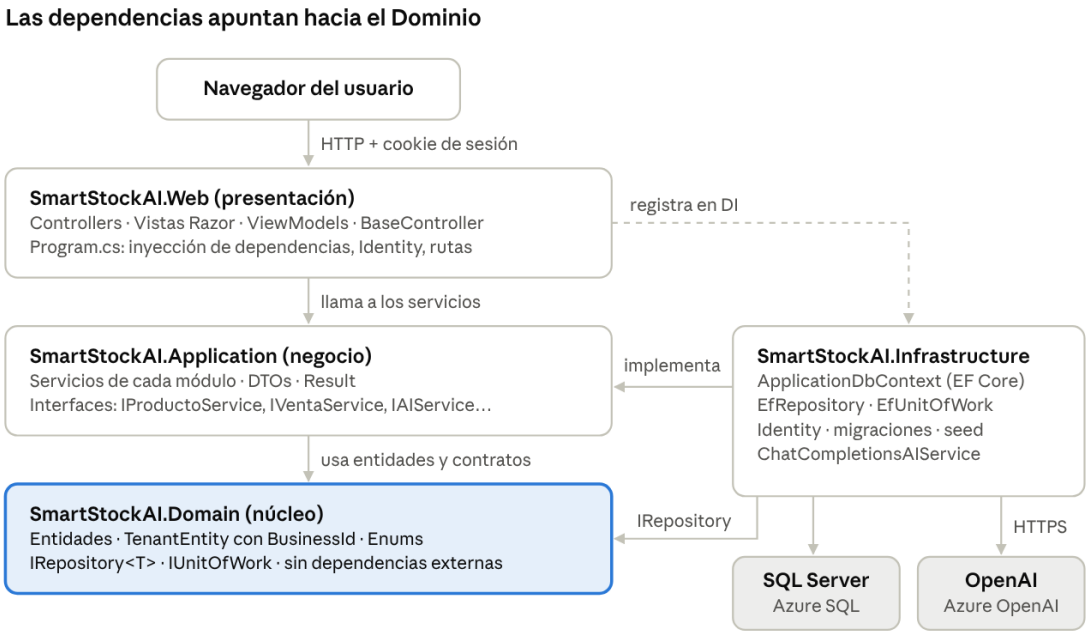
1. Introducción

SmartStock AI es una aplicación web ASP.NET Core 10 MVC para la gestión de inventario, ventas y compras de MYPEs salvadoreñas, con un módulo de recomendaciones basado en inteligencia artificial. Usa Entity Framework Core sobre SQL Server, se organiza en cuatro capas (Clean Architecture) y se ejecuta en contenedores Docker.

Este manual describe cómo está construido el sistema: su arquitectura, las tecnologías empleadas, la estructura del repositorio, las configuraciones realizadas, la integración de IA y la configuración de Docker. Está dirigido a desarrolladores que necesiten mantener, extender o desplegar la aplicación. Los pasos de instalación están en el Manual de Instalación y el uso de las pantallas, en el Manual de Usuario.

2. Arquitectura del sistema

SmartStock AI sigue una arquitectura en capas (Clean Architecture) con cuatro proyectos: Web, Application, Domain e Infrastructure. La capa de presentación usa el patrón MVC de ASP.NET Core, y la lógica de negocio no depende de la base de datos ni del proveedor de IA, sino de interfaces.



Arquitectura en capas · 4 proyectos y 2 servicios externos

Las flechas sólidas indican dependencia: Web llama a Application, Application usa Domain, e Infrastructure implementa las interfaces declaradas en Application y Domain. La flecha

punteada indica que `Program.cs` conecta cada interfaz con su implementación mediante inyección de dependencias.

2.1 Responsabilidad de cada capa

Capa	Contiene	No debe
Web	Controladores, vistas Razor, ViewModels, configuración y arranque.	Contener reglas de negocio ni consultas a la base.
Application	Servicios por módulo (Productos, Ventas, Compras, Inventario, Reportes, Dashboard, Recomendaciones), DTOs, `Result` e interfaces.	Conocer EF Core, SQL Server ni el proveedor de IA.
Domain	Entidades, enums, `IRepository<T>` e `IUnitOfWork`.	Depender de ningún otro proyecto ni framework.
Infrastructure	`ApplicationDbContext`, configuraciones Fluent API, repositorios, Identity, seed y cliente de IA.	Ser referenciada por Application o Domain.

2.2 Flujo de una petición

Por ejemplo, al registrar una venta:

1. El navegador envía el carrito por `fetch` (JSON) a `VentasController`, con el token anti-falsificación en el header.
2. `[Authorize]` verifica la cookie de sesión y `BaseController` obtiene el `BusinessId` del usuario.
3. El controlador llama a `IVentaService` con el DTO de la venta y el `BusinessId`.
4. `VentaService` valida stock y datos, y dentro de `IUnitOfWork.ExecuteInTransactionAsync` crea la venta, sus detalles y los movimientos de inventario, y descuenta el stock.
5. `EfRepository` y `ApplicationDbContext` traducen las operaciones a SQL y las confirman en SQL Server.
6. El servicio devuelve un `Result`; el controlador responde con éxito o con el mensaje de error de negocio.

2.3 Multi-tenencia

Una sola base de datos atiende a varios negocios. Cada fila operativa lleva `BusinessId` y todas las consultas se filtran por el negocio del usuario autenticado (detalles en la sección 5.4).

3. Tecnologías utilizadas

Todo el backend y el frontend están escritos en C# sobre .NET 10; el frontend son vistas Razor renderizadas en el servidor con Bootstrap y algo de JavaScript.

Capa / área	Tecnología	Versión	Uso en el proyecto
Plataforma	.NET / C#	10.0	Todos los proyectos (`net10.0`).
Web	ASP.NET Core MVC + Razor	10.0	Controladores, vistas y enrutamiento.
Acceso a datos	Entity Framework Core (SqlServer, Design, Tools)	10.0.12	ORM, configuraciones Fluent API, migraciones.
Base de datos	SQL Server	2022 (contenedor)	Motor relacional; Azure SQL Database en la nube.
Seguridad	ASP.NET Core Identity + EF Core	10.0.12	Usuarios, roles, contraseñas, bloqueo y cookies.
IA	OpenAI / Azure OpenAI (API Chat Completions)	—	Recomendaciones de inventario vía `HttpClient` tipado.
Interfaz	Bootstrap	5.3.3	Maquetación, modales y componentes.
Interfaz	Chart.js	4.4.4	Gráfica de ventas del Dashboard.
Interfaz	Font Awesome / Google Fonts (Poppins)	6.5.2	Íconos y tipografía.
Pruebas	xUnit, Moq, EF Core SQLite	2.9.2 / 4.20.72 / 10.0.12	Pruebas unitarias e integración (141 casos).
Contenedores	Docker + Docker Compose	—	Imagen multi-etapa de la app y SQL Server.
Nube (propuesta)	Microsoft Azure App Service + Azure SQL	—	Despliegue propuesto para producción.

Las librerías de interfaz se cargan desde CDN (jsDelivr, cdnjs y Google Fonts), por lo que el navegador del usuario necesita acceso a Internet. No se usa ningún SDK de terceros para la IA: la llamada se hace directamente por HTTP con `System.Net.Http.Json`.

4. Estructura del proyecto

El repositorio sigue la estructura pedida por la cátedra: el código en `/src`, la documentación en `/docs`, el esquema de base de datos en `/database` y los archivos de contenedores en `/docker`.

```
SmartStockAI/
├── src/
```

```
│   ├── SmartStockAI.sln
│   ├── SmartStockAI.Domain/          Entidades, enums e interfaces de
repositorio
│   │   ├── Entities/                 BaseEntity, TenantEntity, Business,
Producto, Venta...
│   │   ├── Enums/                    EstadoDocumento, MetodoPago,
TipoMovimientoInventario...
│   │   └── Interfaces/               IRepository<T>, IUnitOfWork
│   ├── SmartStockAI.Application/     Lógica de negocio
│   │   ├── Common/                   Result, PagedResult, AIVocabulary
│   │   ├── DTOs/                     Objetos de transferencia por módulo
│   │   ├── Interfaces/               IProductoService, IVentaService,
IAIService...
│   │   └── Services/                 Implementación de cada servicio
│   ├── SmartStockAI.Infrastructure/  Acceso a datos y servicios externos
│   │   ├── AI/                       AIOptions, ChatCompletionsAIService,
AIResponseParser, InventoryPrompts
│   │   ├── Data/                     ApplicationDbContext, Configurations/,
Seed/, Migrations/
│   │   ├── Identity/                 ApplicationUser, Roles
│   │   └── Repositories/             EfRepository<T>, EfUnitOfWork
│   ├── SmartStockAI.Web/             Presentación (ASP.NET Core MVC)
│   │   ├── Controllers/              Un controlador por módulo +
BaseController
│   │   ├── ViewModels/               Modelos de las vistas
│   │   ├── Views/                    Vistas Razor por módulo y Shared/
_Layout
│   │   ├── wwwroot/                  css, js, img
│   │   ├── appsettings.json
│   │   └── Program.cs                Configuración e inyección de
dependencias
│   └── SmartStockAI.Tests/           Pruebas xUnit (AI, Domain, Services,
Integration)
├── docs/                             Manuales y evidencia de pruebas
├── database/                         schema.sql generado por EF Core +
README
├── docker/                           Dockerfile y docker-compose.yml
├── .env.example                      Variables de entorno sin valores
reales
├── .dockerignore
├── .gitignore
└── README.md
```

4.1 Dependencias entre proyectos

Proyecto	Responsabilidad	Referencia a
`SmartStockAI.Domain`	Núcleo: entidades y contratos. No depende de ningún framework.	—
`SmartStockAI.Application`	Casos de uso, reglas de negocio, DTOs e interfaces de servicios.	Domain

Proyecto	Responsabilidad	Referencia a
`SmartStockAI.Infrastructure`	EF Core, Identity, repositorios y cliente de IA.	Application
`SmartStockAI.Web`	Controladores, vistas, configuración y arranque.	Infrastructure
`SmartStockAI.Tests`	Pruebas de servicios, dominio, parser de IA e integración.	Application, Infrastructure

Las dependencias apuntan siempre hacia el centro: Domain no conoce a nadie y Application solo conoce a Domain. Por eso `IAIService` se declara en Application y se implementa en Infrastructure.

4.2 Modelo de datos

`Business` es el tenant: se relaciona 1—N con `Categoria`, `Proveedor`, `Producto`, `Venta`, `Compra` y `MovimientoInventario`. `Venta` tiene 1—N `VentaDetalle` y `Compra` tiene 1—N `CompraDetalle`. `Producto` pertenece a una `Categoria` y opcionalmente a un `Proveedor`. Las tablas de ASP.NET Identity (`AspNetUsers`, `AspNetRoles`, etc.) conviven en la misma base; `ApplicationUser` agrega `NombreCompleto`, `BusinessId` y `Activo`.

5. Configuraciones realizadas

Toda la configuración se centraliza en `SmartStockAI.Web/Program.cs` y se alimenta de `appsettings.json`, user-secrets (desarrollo) o variables de entorno (Docker y Azure). Ningún secreto se guarda en el repositorio.

5.1 Base de datos y Entity Framework Core

- **Cadena de conexión:** se lee de `ConnectionStrings:DefaultConnection`; si está vacía, de la variable `SMARTSTOCK_CONNECTION_STRING`. Si no existe ninguna, la aplicación se detiene con un mensaje claro.
- **Contexto:** `ApplicationDbContext` hereda de `IdentityDbContext<ApplicationUser>` y registra `UseSqlServer`.
- **Fluent API:** cada entidad tiene su clase en `Data/Configurations/`, aplicadas con `ApplyConfigurationsFromAssembly`. Ahí se definen los montos como `decimal(18,2)`, los índices (por ejemplo, `(BusinessId, Sku)` único en Producto y `(BusinessId, Fecha)` en Venta, Compra y Movimiento) y el comportamiento al borrar: `Restrict` en casi todas las relaciones, `Cascade` de un documento a sus detalles y `SetNull` del proveedor en Producto.
- **Migraciones:** viven en `Infrastructure/Data/Migrations/` y se aplican al arrancar con `Database.MigrateAsync()`. El script idempotente se genera en `database/schema.sql` con `dotnet ef migrations script -i`.
- **Seed:** `DbSeeder` crea los roles, el negocio demo, el usuario administrador y datos de ejemplo la primera vez que arranca.

5.2 Patrones de acceso a datos

- **Repositorio genérico:** `IRepository<T>` (Domain) implementado por `EfRepository<T>` (Infrastructure), con `GetByIdAsync`, `FindAsync`, `AddAsync`, `Update` y `Remove`.
- **Unidad de trabajo:** `IUnitOfWork.ExecuteInTransactionAsync` envuelve en una transacción las operaciones que tocan varias tablas: registrar o anular ventas y compras, movimientos de inventario y creación de productos con stock inicial. Si algo falla, no queda stock a medias.

5.3 Seguridad e Identity

Configuración	Valor
Longitud mínima de contraseña	8 caracteres, con mayúscula, minúscula y dígito
Bloqueo de cuenta	5 intentos fallidos, 10 minutos
Cookie de sesión	8 horas con expiración deslizante
Rutas	`LoginPath = /Account/Login`, `AccessDeniedPath = /Account/AccessDenied`
Roles sembrados	Administrador, Vendedor, Inventario
Anti-falsificación	Token en formularios y en el header `RequestVerificationToken` para las llamadas fetch del carrito

Todos los controladores heredan de `BaseController`, marcado con `[Authorize]`: ninguna pantalla operativa es accesible sin sesión.

5.4 Aislamiento multi-tenant

Cada negocio es un tenant. Las entidades operativas heredan de `TenantEntity`, que agrega `BusinessId`. Al iniciar sesión, `AccountController` guarda el `BusinessId` del usuario como claim en la cookie; `BaseController.CurrentBusinessId` lo lee y cada controlador lo pasa a los servicios, que filtran todas las consultas por ese valor. Así un usuario nunca ve datos de otro negocio.

5.5 Inyección de dependencias

Registro	Ciclo de vida
`IRepository<>` → `EfRepository<>`, `IUnitOfWork` → `EfUnitOfWork`	Scoped
`ICategoriaService`, `IProveedorService`, `IProductoService`, `IInventarioService`, `IVentaService`, `ICompraService`, `IDashboardService`, `IReporteService`, `IRecomendacionService`	Scoped

Registro	Ciclo de vida
`IAIService` → `ChatCompletionsAIService`	`HttpClient` tipado (`AddHttpClient`)
`AIOptions`	`Configure<AIOptions>` desde la sección `AI`

5.6 Variables de entorno

Variable	Uso
`ConnectionStrings__DefaultConnection`	Cadena de conexión (la usa Docker Compose).
`SMARTSTOCK_CONNECTION_STRING`	Alternativa para Azure App Service.
`ASPNETCORE_ENVIRONMENT`	`Development` o `Production`.
`DB_SA_PASSWORD`	Contraseña del usuario `sa` del contenedor SQL Server.
`AI_PROVIDER`, `AI_MODEL`, `AI_API_KEY`, `AI_ENDPOINT`, `AI_API_VERSION`	Configuración de IA; Compose las pasa como `AI__Provider`, `AI__Model`, etc.

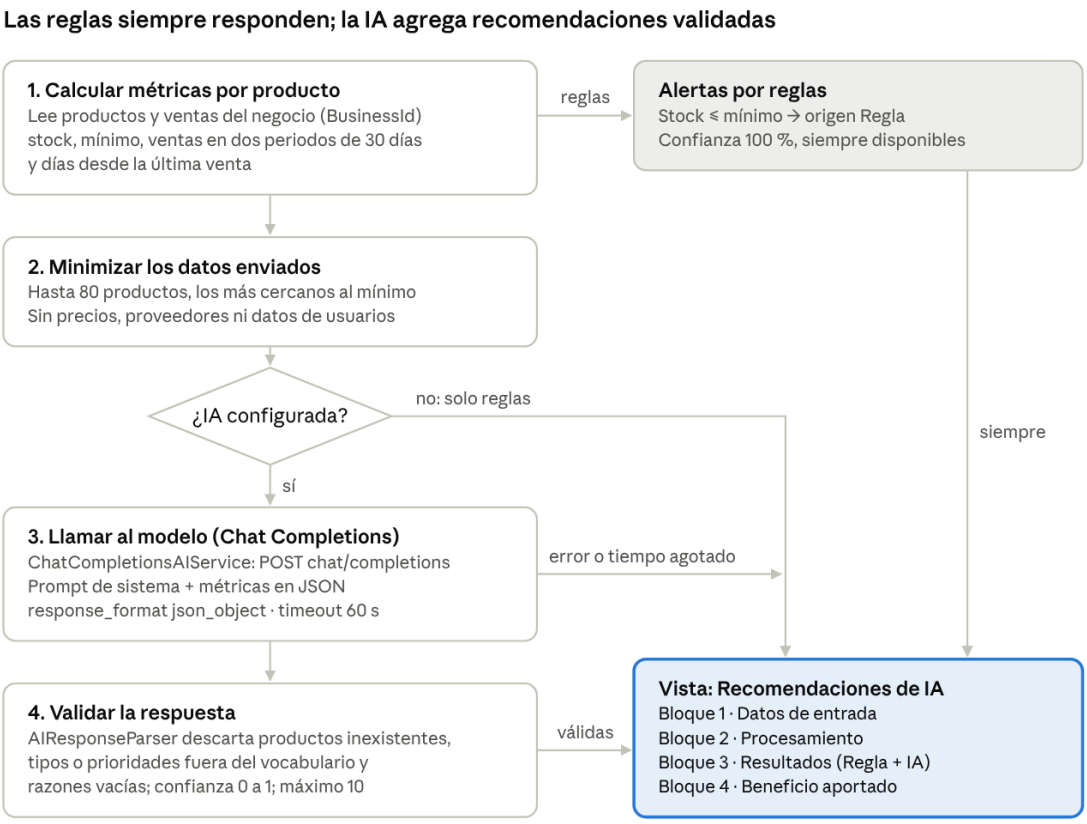
`.env.example` documenta estas variables sin valores reales; `.env` está en `.gitignore`. En desarrollo local los secretos se guardan con `dotnet user-secrets`.

5.7 Manejo de errores y validaciones

- **Errores no controlados:** en desarrollo se muestra `UseDeveloperExceptionPage`; en producción, `UseExceptionHandler("/Home/Error")` y HSTS, sin exponer detalles técnicos.
- **Errores de negocio:** los servicios devuelven `Result` / `Result<T>` en lugar de lanzar excepciones (por ejemplo, «stock insuficiente»), y el controlador lo muestra como mensaje en la vista.
- **Validaciones:** anotaciones en ViewModels y DTOs, comprobación de `ModelState` en los controladores y reglas de negocio en los servicios (SKU único por negocio, cantidades positivas, stock suficiente).
- **HTTPS:** fuera de contenedor se usa `UseHttpsRedirection`; dentro del contenedor el TLS lo termina el proxy (Azure App Service), por lo que la app solo escucha HTTP en el puerto 8080.
- **Salud:** el endpoint `GET /health` responde `ok` y sirve para el health check de Azure App Service.

6. Integración de IA

La funcionalidad de IA es un **sistema de recomendaciones de inventario**: un modelo de lenguaje (OpenAI o Azure OpenAI) recibe métricas ya calculadas por producto y devuelve sugerencias en JSON, que el sistema valida antes de mostrarlas. Se combina con reglas determinísticas, de modo que el módulo sigue siendo útil aunque la IA no esté disponible.



Flujo del análisis de inventario · 4 pasos, 1 decisión, 2 caminos de respaldo

Si la IA no está configurada o falla la llamada, la vista muestra solo las alertas por reglas y un mensaje explicando por qué; la aplicación nunca se cae por un error del proveedor.

6.1 Componentes

Componente	Capa	Responsabilidad
`IAIService`	Application	Contrato: `AnalyzeInventoryAsync`, `ProviderName`, `ModelName`, `IsConfigured`.
`RecomendacionService`	Application	Calcula métricas, genera alertas por reglas, selecciona los productos a enviar, llama a la IA y ordena el resultado por prioridad.
`AIVocabulary`	Application	Tipos y prioridades permitidos.
`ChatCompletionsAIService`	Infrastructure	Cliente HTTP tipado que llama a la API Chat Completions.
`InventoryPrompts`	Infrastructure	Prompt de sistema con las reglas que debe seguir el modelo.
`AIResponseParser`	Infrastructure	Convierte y valida el JSON devuelto.

Componente	Capa	Responsabilidad
`AIOptions`	Infrastructure	Configuración: `Provider`, `Model`, `ApiKey`, `Endpoint`, `ApiVersion`.
`RecomendacionesController` + vista	Web	Ejecuta el análisis al pulsar **Analizar inventario** y muestra los cuatro bloques.

6.2 Configuración del proveedor

`AIOptions` se carga desde la sección `AI` de la configuración (user-secrets en desarrollo, variables `AI__*` en Docker o Azure). En `appsettings.json` la sección existe pero sin valores.

Proveedor	`BaseAddress` del `HttpClient`	Autenticación	Modelo
`OpenAI`	`https://api.openai.com/v1/`	Header `Authorization: Bearer <ApiKey>`	En el cuerpo (`model`).
`AzureOpenAI`	`<Endpoint>/openai/deployments/<Model>/` + `?api-version=<ApiVersion>`	Header `api-key: <ApiKey>`	Es el nombre del deployment, va en la URL.

La IA se considera configurada cuando hay `ApiKey` y `Model` (y, para Azure, también `Endpoint` y `ApiVersion`). El `HttpClient` tiene un timeout de 60 segundos.

6.3 Prompt y formato de respuesta

El prompt de sistema define al modelo como analista de inventario para una MYPE salvadoreña y le impone reglas estrictas:

- Usar únicamente los datos recibidos; no inventar productos, cifras, precios ni proveedores.
- Omitir un producto si no hay información suficiente.
- No hacer cálculos financieros y redactar como sugerencias («considerar…», «revisar…»), no como órdenes.
- Citar en cada razón las métricas que la justifican.
- Devolver como máximo 10 recomendaciones, ordenadas por prioridad, en español.

La petición usa `response_format: json_object` y el modelo debe responder solo con este formato:

```
{"recommendations":[{"productId":0,"type":"LOW_STOCK","priority":"HIGH",
  "recommendation":"...","reason":"...","confidence":0.0}]}
```

Tipos permitidos: `LOW_STOCK`, `HIGH_DEMAND`, `LOW_ROTATION`, `PURCHASE_RECOMMENDATION`, `SALES_TREND`, `GENERAL_ALERT`. Prioridades: `HIGH`, `MEDIUM`, `LOW`.

6.4 Validación y manejo de errores

`AIResponseParser` no confía en la respuesta del modelo. Quita cercas de código si las hay, deserializa y conserva solo las recomendaciones cuyo `productId` esté entre los enviados, cuyo tipo y prioridad pertenezcan al vocabulario y que traigan razón y recomendación. La confianza

se limita al rango 0–1 y se toman como máximo 10. Un JSON inválido produce una lista vacía, no una excepción.

`ChatCompletionsAIService` captura errores HTTP, tiempos agotados y JSON mal formado. El detalle técnico se registra en el log y al usuario solo le llega un mensaje genérico. Las pruebas de `AIResponseParser` están en `SmartStockAI.Tests/AI`.

6.5 Evidencia: entrada, procesamiento, resultados y beneficio

Elemento pedido	Cómo lo evidencia el sistema
Datos de entrada	Bloque 1 de la vista: tabla con producto, categoría, stock, mínimo, ventas de los últimos 30 días, de los 30 anteriores y días sin venta, exactamente lo que se envió.
Procesamiento	Bloque 2: reglas aplicadas, proveedor y modelo, estado de la IA y duración en milisegundos (medida con `Stopwatch`).
Resultados	Bloque 3: alertas por reglas y recomendaciones de IA con prioridad, tipo, confianza, recomendación, razón y origen.
Beneficio	Bloque 4: anticipar quiebres de stock, detectar cambios de demanda y productos de baja rotación, y decidir compras con datos del propio negocio.

6.6 Seguridad y privacidad

La clave de la API nunca se guarda en el repositorio. Al proveedor solo se envían métricas de inventario de hasta 80 productos, sin precios, proveedores ni datos personales. La IA no escribe en la base de datos: sus resultados solo se muestran como sugerencias.

7. Configuración Docker

La solución corre en dos contenedores definidos en `docker/docker-compose.yml`: `smartstock-db` con SQL Server 2022 y `smartstock-web` con la aplicación, construida con un Dockerfile multi-etapa. La web espera a que la base esté sana antes de arrancar y aplica las migraciones al iniciar.

7.1 Dockerfile (multi-etapa)

```
# ---------- Etapa 1: build ----------
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/SmartStockAI.Domain/*.csproj         SmartStockAI.Domain/
COPY src/SmartStockAI.Application/*.csproj    SmartStockAI.Application/
COPY src/SmartStockAI.Infrastructure/*.csproj SmartStockAI.Infrastructure/
COPY src/SmartStockAI.Web/*.csproj            SmartStockAI.Web/
RUN dotnet restore SmartStockAI.Web/SmartStockAI.Web.csproj
COPY src/ .
RUN dotnet publish SmartStockAI.Web/SmartStockAI.Web.csproj \
    -c Release -o /app/publish --no-restore -p:UseAppHost=false

# ---------- Etapa 2: runtime ----------
```

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
USER $APP_UID
ENV ASPNETCORE_HTTP_PORTS=8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "SmartStockAI.Web.dll"]
```

Decisión	Motivo
Dos etapas (`sdk` → `aspnet`)	La imagen final solo lleva el runtime y los binarios publicados, no el SDK ni el código fuente; pesa mucho menos.
Copiar primero los `.csproj` y hacer `restore`	Docker guarda en caché esa capa; si solo cambia el código, no vuelve a descargar paquetes.
`USER $APP_UID`	La aplicación no se ejecuta como root dentro del contenedor.
Solo HTTP en el puerto 8080	El TLS lo termina el proxy (Azure App Service); en local se usa HTTP.
Contexto de build en la raíz del repo	Permite copiar `src/` completo; `.dockerignore` excluye `bin/`, `obj/`, `.git` y `.env`.

7.2 docker-compose.yml

Servicio	Imagen / build	Puertos	Configuración clave
`db` (`smartstock-db`)	`mcr.microsoft.com/mssql/server:2022-latest`	`127.0.0.1:1433:1433` (solo local)	`MSSQL_SA_PASSWORD` desde `DB_SA_PASSWORD`; volumen `sqldata`; healthcheck con `sqlcmd` cada 10 s.
`web` (`smartstock-web`)	Build de `docker/Dockerfile`	`8080:8080`	`depends_on: db (service_healthy)`; `ASPNETCORE_ENVIRONMENT=Production`; cadena de conexión a `Server=db,1433`; variables `AI__*`.

Ambos servicios usan `restart: unless-stopped`. El volumen `sqldata` conserva los datos entre reinicios. La variable `DB_SA_PASSWORD` es obligatoria: si falta en `.env`, Compose se detiene con un mensaje.

7.3 Comandos

```
# Preparar variables (completar DB_SA_PASSWORD y, si se tiene clave, AI_*)
cp .env.example .env
```

```
# Construir y levantar todo
docker compose --env-file .env -f docker/docker-compose.yml up --build

# Ver estado, logs e imágenes
docker compose -f docker/docker-compose.yml ps
docker compose -f docker/docker-compose.yml logs web
docker images

# Solo la base de datos, para desarrollar con dotnet run
docker compose --env-file .env -f docker/docker-compose.yml up db -d

# Detener (agregar -v borra también los datos)
docker compose -f docker/docker-compose.yml down
```

Con los contenedores arriba, la aplicación queda en `http://localhost:8080` con el usuario demo `administrador@smartstock.ai` / `SmartStock2026!`.

7.4 Problemas frecuentes

Síntoma	Causa y solución
`smartstock-db` se reinicia en bucle	La contraseña no cumple la política de SQL Server. Usar una más fuerte y ejecutar `docker compose down -v`.
`Login failed for user 'sa'`	El volumen conserva la contraseña original. Si se cambió en `.env`, ejecutar `docker compose down -v` (borra los datos).
`web` falla al conectar	La contraseña contiene `;`, `=` o `$`, que rompen la cadena de conexión.
Puerto 8080 o 1433 ocupado	Cambiar el puerto de la izquierda en `ports:`, por ejemplo `"8081:8080"`.
Se cierra la sesión al reiniciar el contenedor	Esperado: las llaves de cookies no se persisten. No afecta la demo.

7.5 Nota de seguridad

El seed crea un administrador con contraseña conocida. Es aceptable para la demostración académica, pero en un despliegue real se debe cambiar esa contraseña o deshabilitar el seed.